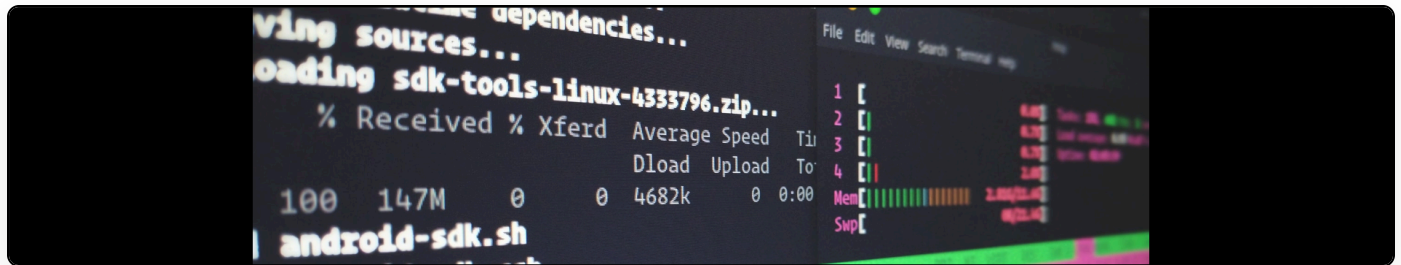


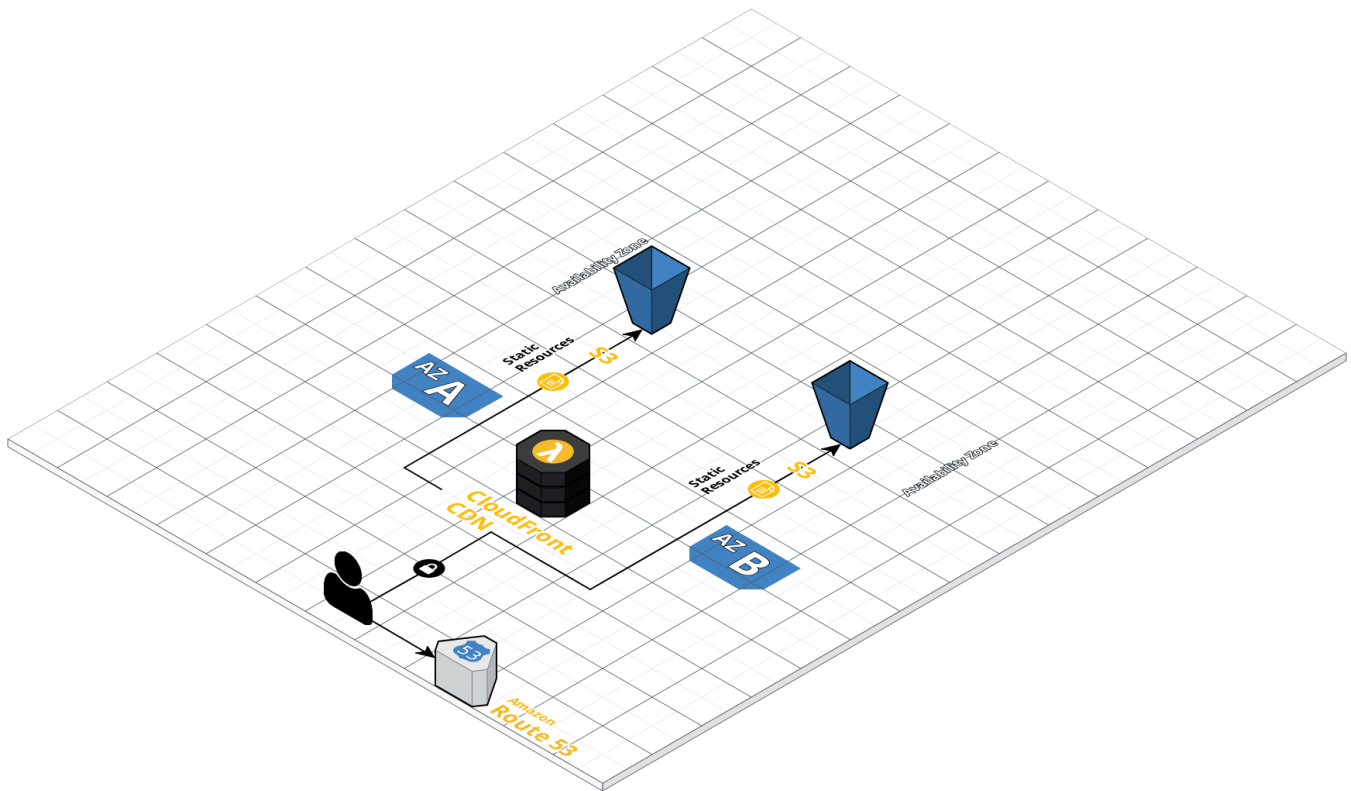
Static site hosting in the cloud should not be this hard in 2022

Cite as: Martin Paul Eve, "Static site hosting in the cloud should not be this hard in 2022", *eve.gd: Martin Paul Eve*, May 07, 2022, <https://doi.org/10.59348/9x33x-1ky82>.



Last weekend I converted my website hosting to an infrastructure-as-code solution. It's no big deal, I thought. It's just a static site so it must be really easy to provision this. Surely just some kind of AWS S3 bucket associated with a custom domain? I mean, generated static sites are great. They are fast, lightweight, and virtually un-hackable. Surely it must be easy to deploy this?

It turns out that hosting a static site on AWS in 2022 is still somewhat painful. The major problem is that you can't serve directly from an S3 bucket using SSL. This means that you have to setup a Cloudfront distribution in front of your static site. Here's a cloudcraft.co diagram of [roughly what I came up with](#).



Some of the snags I hit along the way:

1. You need a Lambda@Edge function to handle redirects. But not just complex redirects. Even really, really basic redirects. Indeed, Cloudfront won't redirect to index.html inside sub-directories where it exists. The official solution is to [write your own function to handle this](#). This function has to be located in the us-east-1 region or it won't work, for some reason. So to get basic directory handling, you already have to write some server-side code and work out how to deploy this, which turns out, also, to be [pretty complicated](#). Debugging these functions is also really difficult, because the Cloudwatch logs replicate to the region where the function ran, not to the region where the function is hosted (which, again, has to be us-east-1). So you end up delving around for ages trying to work out what's going wrong. You need, also, to read and understand the [event lifecycle](#) for Lambda@Edge functions (I needed "origin-request" in the end). This is really not a good solution when, really, you just want is the equivalent of a .htaccess file.

2. Certificate verification is a total pain. I have to manage several SSL certificates because I have subdomains, like books.eve.gd. When you verify these, you have to go in and add the DNS entries/click the email verification link. This felt really fragile/breakable.
3. Route 53 management for DNS with Terraform *is* pretty cool, but it also has loads of snags/gotchas. While most domain registrars and zone hosts use “@” to refer to the root, AWS doesn’t. Hence, in Route 53, you don’t set the name to “@” but to the domain itself. You also can’t add multiple records with the same name – as you would elsewhere – but have to set multiple record entries in a single “aws_route53_record”.

AWS infrastructure (and Terraform for provisioning) is really flexible. You get a huge amount of control. But, really, this should have been *far* easier than it was. I just wanted to chuck my static site into an S3 bucket and get it serving. Instead, it turned into a Cloudfront-S3-Lambda@Edge-Route 53-ACM project.